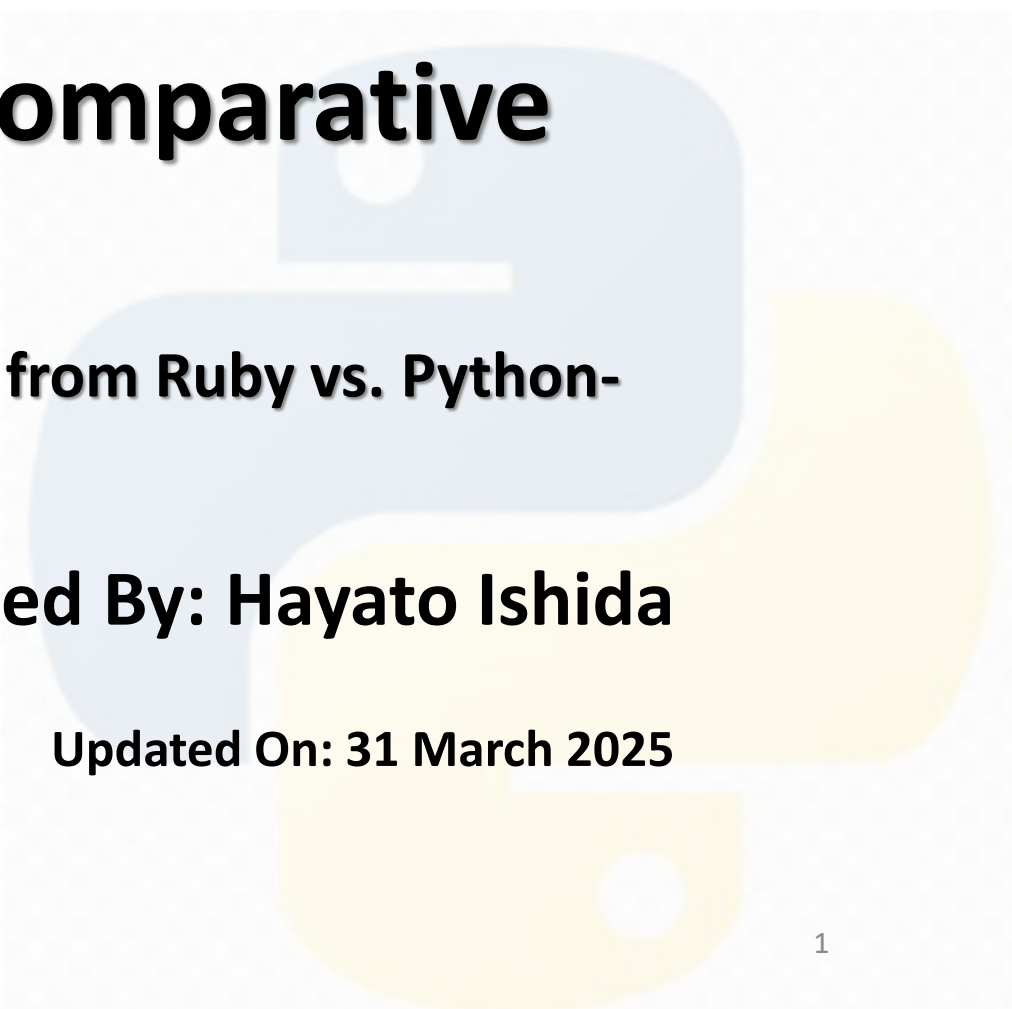




Recommendation of Comparative Linguistics

-Learn Identity of Programming Language from Ruby vs. Python-

Composed By: Hayato Ishida

For Japanese version, click [here](#)

Updated On: 31 March 2025

About Me

- Accounts
 - GitHub: [@hayat01sh1da](#)
 - X: [@hayat01sh1da](#)
 - LinkedIn: [@hayat01sh1da](#)
 - Speaker Deck: [@hayat01sh1da](#)
 - Docswell: [@hayat01sh1da](#)
- Occupation: Software Engineer
- Things I Am Into
 - Linguistics
 - Singing at Karaoke
 - Listening to Music
 - Watching Movies
 - Cat Feeding



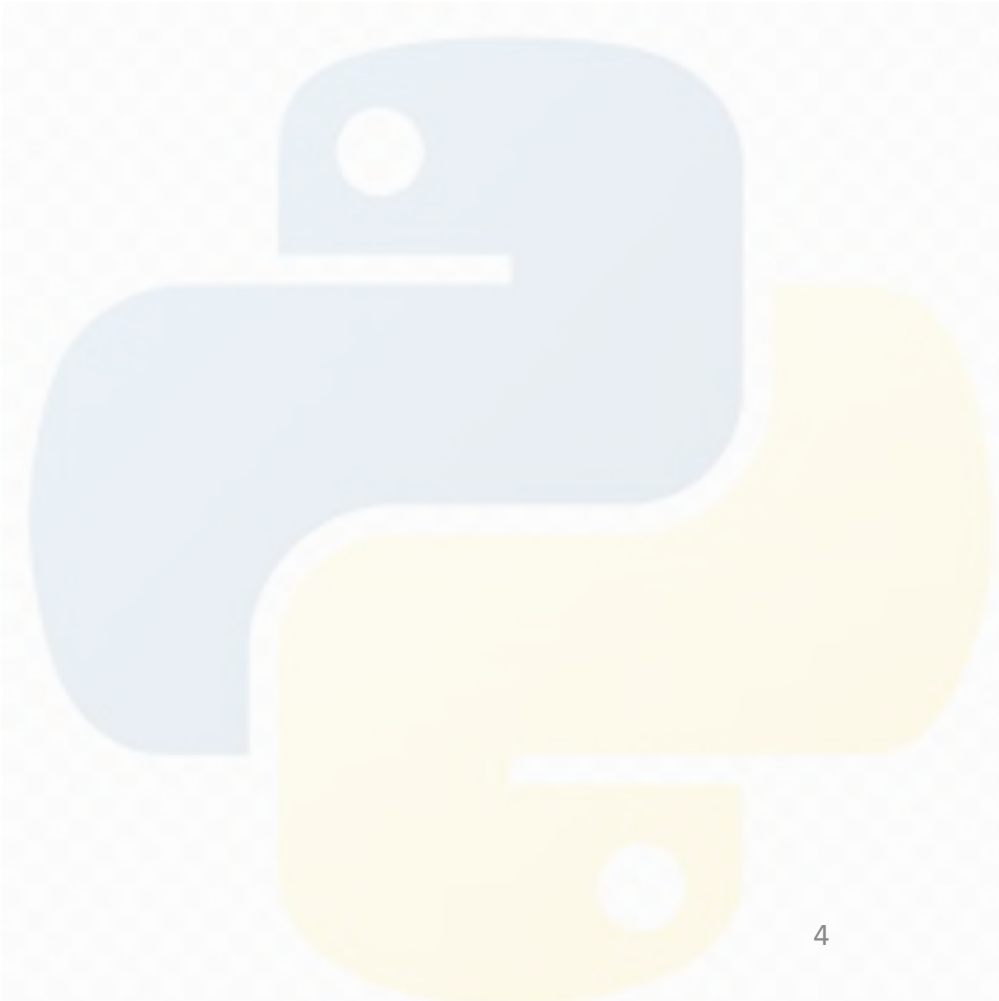
Licences / Certifications

- English
 - TOEIC® Listening & Reading 915: Certified on December 2019
- Engineering
 - Information Security Management: Certified on November 2017
 - Applied Information Technology Engineer: Certified on June 2017
 - Fundamental Information Technology Engineer: Certified on November 2016
 - IT Passport: Certified on April 2016
- Others
 - Abacus 2nd Class: Certified on June 2002
 - Mental Arithmetic 3rd Class: Certified on February 2001

Skill Stack

- Languages
 - Japanese: Native Proficiency
 - English: Full professional Proficiency
- Development
 - **Backend: Ruby(Ruby on Rails), Python**
 - Frontend: TypeScript + React.js, TypeScript +Vue.js
 - **Database: MySQL**, PostgreSQL, MongoDB
 - **Architecture: Monolith, Modular Monolith**
 - Hosting: AWS ESK
 - Versioning: Git, GitHub
 - **CI/CD: GitHub Actions**, ArgoCD
 - Monitoring: Datadog, Sentry

VS.



Work Histories

Period	Genre	Position	Jobs
Aug 2021 - Present	SaaS(Educational Service)	Software Engineer	• Backend Development • Frontend Development • CI/CD Maintenance • Weekly Release Manager • Troubleshooting • Documentation • Technical Debt Resolution • Tech Blog Entries • LT Speaker
Feb 2020 - Dec 2020	SaaS(Chatbot Platform)	Backend Engineer	• Backend Development • Documentation • Technical Debt Resolution • Inspection of Alternative API

Work Histories

Period	Genre	Position	Jobs
Jun 2018 - Jan 2020	Contract-basis Developer	Software Engineer	• Backend Development • Frontend Development • Troubleshooting • Quality Assurance • Tech Blog Entries
Apr 2016 - Jan 2018	System Engineering Service	System Engineer	• Corporate Account Admin • Windows Server Maintenance • Security Admin Assistant • Internationalisation / Localisation

International-Exchange Activities

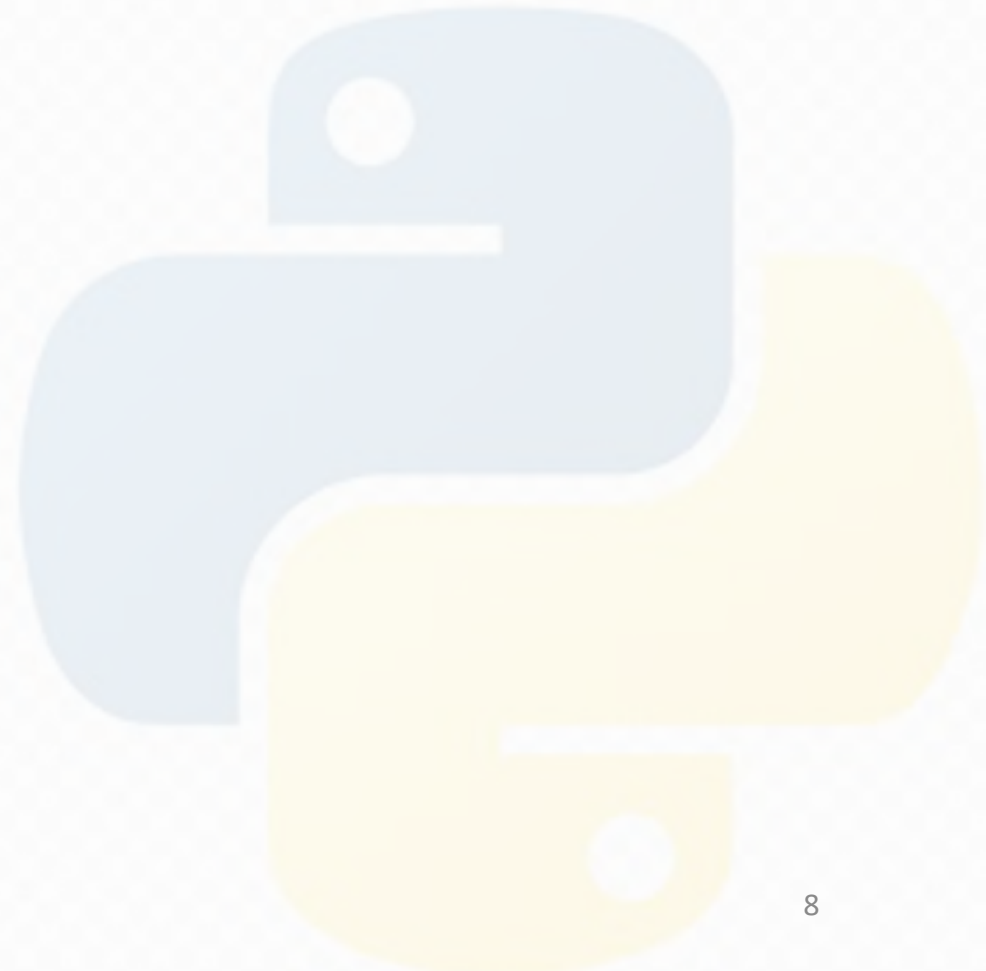
- Activities in University
 - English Linguistics Seminar(Focusing on Mass Media English)
 - International-Exchange Clubs(The 2nd Year)
 - [International-Exchange Programmes conducted by Japan Cabinet Office](#)(2013 - 2016)
 - Japanese Linguistics Course(The Final Year)
- Overseas Life Experience
 - Working Holiday in Australia(April 2014 - March 2015)
 - Language School for 1 month in Sydney
 - Work for 6 Months in [Hamilton Island Resort](#)
 - Volunteering for 1 Month as Assistant Teacher of Japanese Language at [St Ives High School](#) in NSW
- Other Activities
 - Keep Everyday Journal in English (April 2014 - Present)
 - [Sunrise Toastmasters Club](#)(February 2017 - March 2018)
 - [Vital Japan](#)(January 2018 - July 2019, October 2022 - February 2023)
 - Self Learning and Training of English Language
 - Participation Ruby-Related Tech Conferences

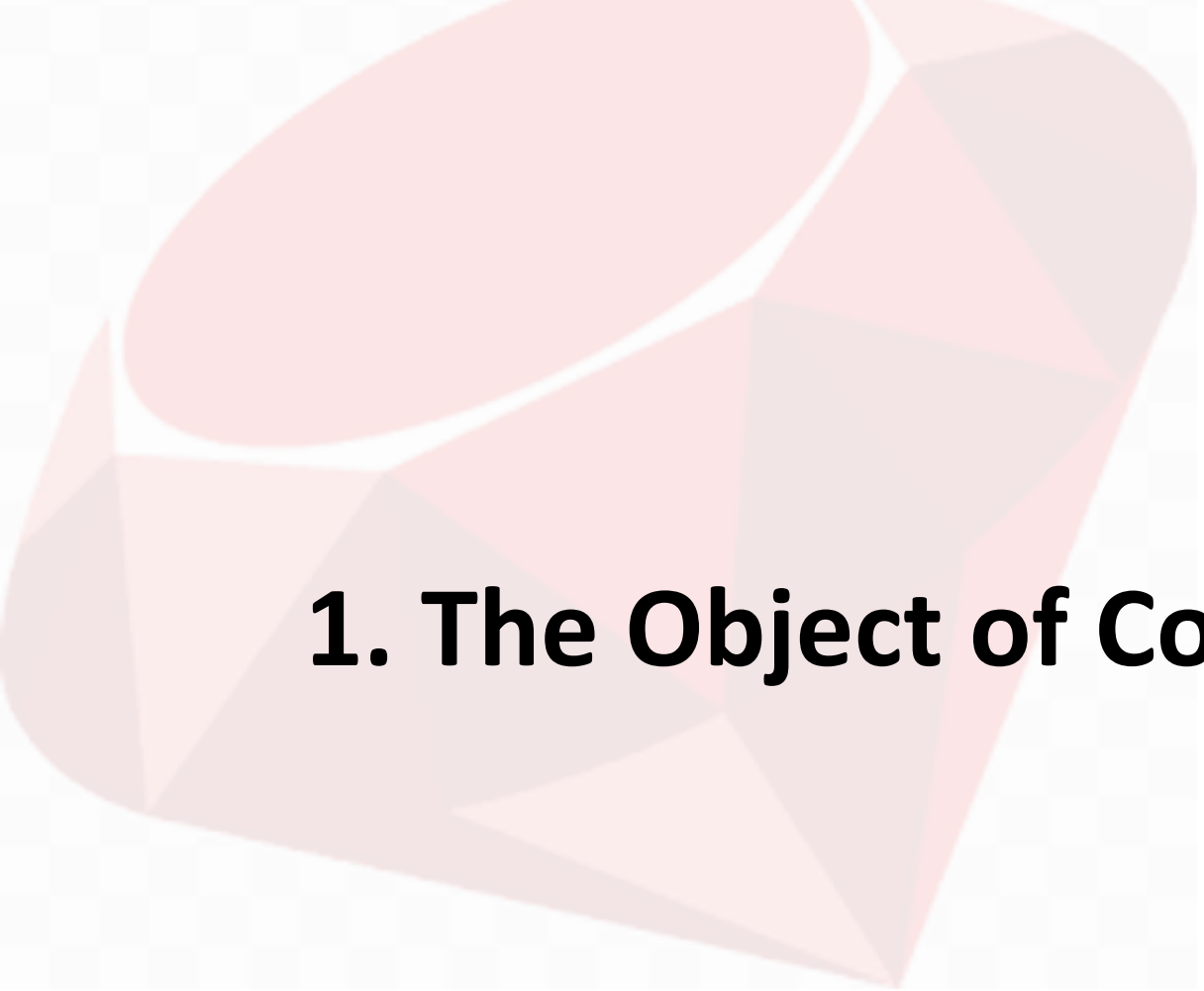
VS.

Agenda

1. [The Object of Comparative Linguistics](#)
2. [3 Common Features between Ruby and Python](#)
3. [3 Different Features between Ruby and Python](#)
4. [Epilogue](#)
5. [References](#)

VS.



A large, semi-transparent pink geometric shape composed of several overlapping triangles and polygons, resembling a stylized leaf or a modern logo, set against a light gray and white checkerboard background.

1. The Object of Comparative Linguistics

vs.

The Python logo, consisting of two interlocking snakes, one blue and one yellow, set against a light gray and white checkerboard background.

1. The Object of Comparative Linguistics

I majored in English linguistics in university.

The biggest lesson was that **comparison of one with another gives us insight into the essence of an object.**

The following saying vividly represents it. **vs.**

"He who knows no foreign language knows nothing of his mother tongue." by Johann Wolfgang von Goethe, the greatest writer in the Germanic language.

1. The Object of Comparative Linguistics

As for natural languages, we are forced to be aware of the differences between the mother tongue and another language when we command any in learning, life, and at work, which provides us with deep insight into the identity of us and our language.

VS.

✂ For the details, refer to [*Dive Deeper into Present-Day English Vol.5 -Hidden Feature of Language-*](#).

1. The Object of Comparative Linguistics

Regarding programming languages, Mr. Anders Norås mentions almost the same experience on [97 Things Every Programmer Should Know > 27. Don't Just Learn the Language, Understand its Culture.](#)

*In their seminal book, The Pragmatic Programmer, Andy Hunt and Dave Thomas encourage us to learn a new programming language every year. I've tried to live by their advice and throughout the years I've had the experience of programming in many languages. My most important lesson from my polyglot adventures is that **it takes more than just learning the syntax to learn a language: You need to understand its culture.***

1. The Object of Comparative Linguistics

Nowadays, width and depth of responsibility of software engineers gets so greater that we cannot help feeling like being able to command multiple numbers of programming language is the de facto standard, as it were.

Such requirement supplies **more deep understanding of the first acquired language than skillset itself.**

LT physically restricts the upper limit of allocated time, so I focus on **pulling a trigger of awareness of what Ruby is**, my (and audience's) main programming language.



2. 3 Common Features between Ruby and Python

vs.

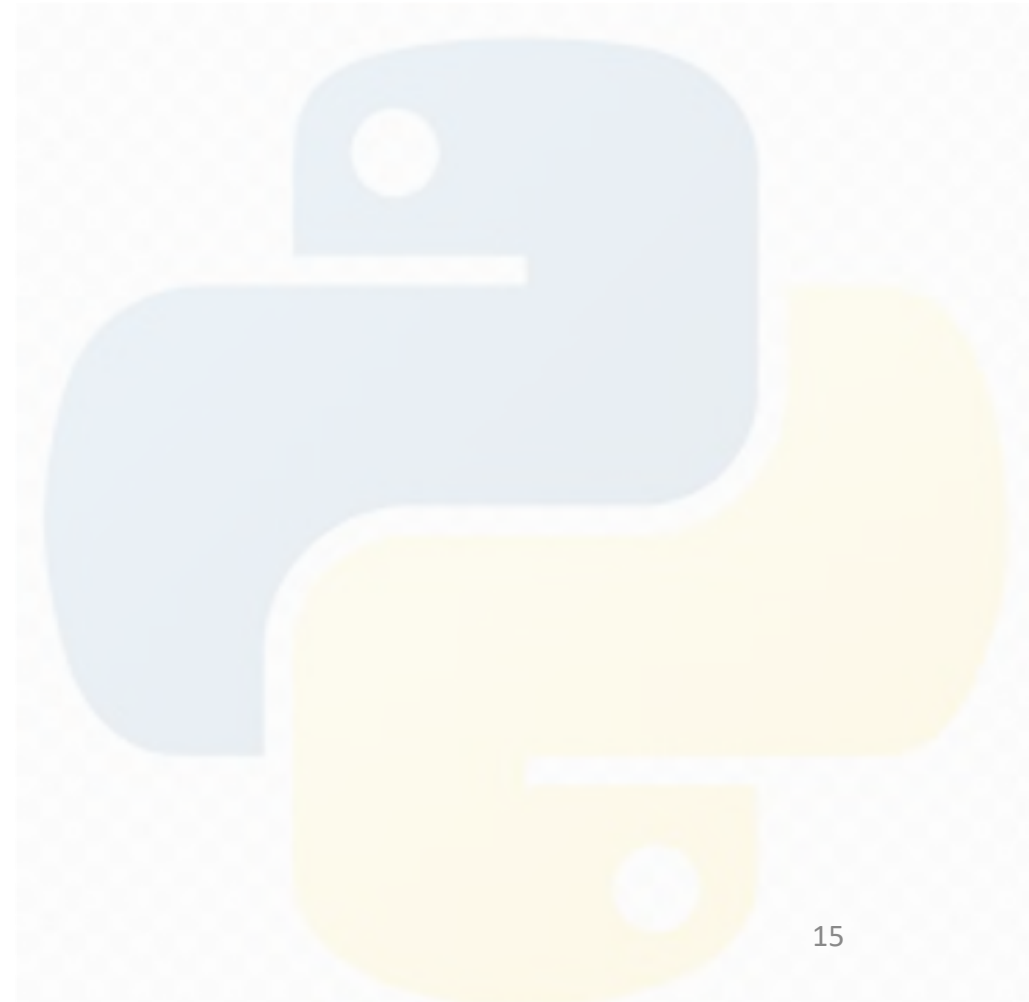


2. 3 Common Features between Ruby and Python

Ruby and Python share the following 3 features.

1. [Object-Oriented](#)
2. [Dynamic Typing](#)
3. [Values Developer's Experience](#)

VS.



2. 3 Common Features between Ruby and Python

① Object-Oriented

There are the following 3 principles.

1. **Inheritance:** Subclass takes over abstract methods and attributes from the base class. Subclass inherits shared nature of the base class, which allows developers to implement only the differences. Thanks to this principle, codes are so reusable that developers can program in an efficient way.
2. **Polymorphism:** Behaviour differs from receiver to receiver even if the name of the called method is the same (e.g., `Doctor#practice` means starting a medical business, whereas `Athlete#practice` performing again and again in order to acquire a specific physical skill).
3. **Encapsulation:** The design that developers can be concerned only about the outer behaviours by communication of messages between receivers, method calls and accesses to attributes, with the inner detailed data processing and sustained data hidden. It prevents deep dependencies on the inner processes at the level of requirements, which makes sure of maintainability.

2. 3 Common Features between Ruby and Python

② Dynamic Typing

If the data type is determined **during compilation in static typing** while it is fixed **at runtime in dynamic typing**.

C, Java and TypeScript belong to the former, and Ruby, Python, JavaScript and PHP the latter.

In dynamic typing such as Ruby and Python, **developers are not allowed to interfere in typing so that what data is sustained in the variable stays unknown till the execution.**

That is why **understandable naming of variables at the first sight is highly important** particularly in those languages.

Mr. Yukihiro Matsumoto, called Matz and the creator of Ruby, mentions the fact that **appropriate naming reflects the corresponding appropriate design** in [*97 Things Every Programmer Should Know > 107. The Importance of Naming*](#), Japanese edition.

2. 3 Common Features between Ruby and Python

③ Values Developer's Experience

Ruby and Python provide developers with wonderful coding experiences via ease of execution and dynamic programming(e.g., ``eval`` and ``Module#class_eval``).

Even roughly written program runs most of time, so **the developer who has poor skills, experience and understanding is highly likely to undergo bad readability and hardship of tough debugging, not just by themselves but those concerned.**

Carefully design the system based on the requirements, implement necessary classes and methods with the least scope of responsibility.

You must be sure how important testing codes are if you are following what the state above means.



3. 3 Different Features between Ruby and Python

vs.



3. 3 Different Features between Ruby and Python

Ruby and Python have the following 3 differences.

1. [Presence of Concept of Private Field](#)
2. [The Way to Cache Return Value via Delayed Evaluation](#)
3. [Receiver and Argument](#)

3. 3 Different Features between Ruby and Python

① Presence of Concept of Private Field

Ruby allows developers to define the private field with `private` keyword.

In contrast, Python has no concept of private field itself.

vs.

3. 3 Different Features between Ruby and Python

① Presence of Concept of Private Field

```

1 class User < You, 25 minutes ago • Commit sample code in Ruby
2   def initialize(name:, age:, occupation:, favourites: [])
3     @name = name
4     @age = age
5     @occupation = occupation
6     @favourites = favourites
7   end
8
9   def greet
10    "Hi, I'm #{@name} and #{@age} years old. \nMy occupation is #{@occupation}. \nMy favourite-#{inflect('language')} is #{@favourite_programming_languages}."
11  end
12
13  private
14
15  attr_reader :name, :age, :occupation, :favourites
16
17  def favourite_programming_languages
18    @favourite_programming_languages ||= favourites.select { |it| it[:genre].equal?(:programming) }
19  end
20
21  def favourite_programming_language_names
22    return 'nothing' if favourite_programming_languages.empty?
23    favourite_programming_languages.map { |it| it[:language] }.insert(-2, 'and').join(', ').gsub(/and\./, ' and ')
24  end
25
26  def inflect(obj)
27    case obj
28    when 'be'
29      (0..1).include?(favourite_programming_languages.length) ? 'is' : 'are'
30    when 'language'
31      (0..1).include?(favourite_programming_languages.length) ? obj : "#{obj}s"
32    end
33  end
34 end
35
36 favourites = [
37   { genre: :programming, language: 'Ruby', level: 'Upper-Intermediate', years_of_experience: 7 },
38   { genre: :programming, language: 'Python', level: 'Intermediate', years_of_experience: 7 },
39   { genre: :programming, language: 'JavaScript', level: 'Intermediate', years_of_experience: 7 }
40 ]
41
42 user = User.new(name: 'Hayato Ishida', age: 33, occupation: 'Software Engineer', favourites: favourites)
43 puts user.greet

```

```

1 import re
2
3 You: 2 minutes ago | 1 author (You)
4 class User:
5     def __init__(self, name, age, occupation, favourites = []):
6         self.name = name
7         self.age = age
8         self.occupation = occupation
9         self.favourites = favourites
10        self.favourite_programming_languages = self._favourite_programming_languages_()
11
12    def greet(self):
13        return "Hi, I'm {name} and {age}-years-old.\nMy occupation is {occupation}.\nMy favourite {language} {be_verb} {favourite_programming_language_names}.".format(
14            name=self.name,
15            age=self.age,
16            occupation=self.occupation,
17            language=self._infect_('language'),
18            be_verb=self._infect_('be'),
19            favourite_programming_language_names=self._favourite_programming_language_names_())
20
21    @private
22    def _favourite_programming_languages_(self):
23        self.favourite_programming_languages = []
24
25        for favourite in self.favourites:
26            if favourite['genre'] == 'programming':
27                self.favourite_programming_languages.append(favourite)
28
29        return favourite_programming_languages
30
31    def _favourite_programming_language_names_(self):
32        if len(self.favourite_programming_languages) == 0:
33            return 'nothing'
34
35        favourite_programming_language_names = []
36
37        for favourite_programming_language in self.favourite_programming_languages:
38            favourite_programming_language_names.append(favourite_programming_language['language'])
39
40        favourite_programming_language_names.insert(-1, 'and')
41
42        return re.sub(r'and,', ', and', ','.join(favourite_programming_language_names))
43
44    def _infect_(self, obj):
45        match obj:
46            case 'be':
47                if len(self.favourite_programming_languages) in range(0, 2):
48                    return 'is'
49                else:
50                    return 'are'
51            case 'language':
52                if len(self.favourite_programming_languages) in range(0, 2):
53                    return obj
54                else:
55                    return '{obj}s'.format(obj=obj)
56
57 if __name__ == '__main__':
58     favourites = [
59         {'genre': 'programming', 'language': 'Ruby', 'level': 'Upper-Intermediate', 'years_of_experience': 7},
60         {'genre': 'programming', 'language': 'Python', 'level': 'Intermediate', 'years_of_experience': 7},
61         {'genre': 'programming', 'language': 'JavaScript', 'level': 'Intermediate', 'years_of_experience': 7}
62     ]
63
64     user = User(name='Hayato Ichida', age=33, occupation='Software Engineer', favourites=favourites)

```

3. 3 Different Features between Ruby and Python

① Presence of Concept of Private Field

As mentioned in [2. 3 Common Features between Ruby and Python > ① Object-Oriented > 3. Encapsulation](#), **Object-Oriented programming languages normally have system which conceal the detailed inner data processing and sustained data, but Python does not.**

This allows method calls and override of data even outside of the class with their definition, which is terribly vulnerable to security risks and bugs.

Also, ``def _method`` and ``def __method__`` implicitly enables developers to tell public methods from private ones; however, it is not guaranteed by the Python's paradigm, so this means is not safe at all.

What is worse, methods provided by libraries often start with ``_``, so the way shown above can unexpectedly override the library method.

This is reported on X and Quora as far as I observed.

3. 3 Different Features between Ruby and Python

② The Way to Cache Return Value via Delayed Evaluation

What symbols are available totally depends on the programming language.

``||`` is one of the familiar examples.

Ruby utilises this symbol for memoization, caching in other words.

On the contrary, ``||`` is not available in Python, so it does not provide memoization in a method definitions via delayed evaluation as Ruby does.

Python is likely to allow us only to cache data by assignment of a return value of a method to a instance variable as show on the next slide.

3. 3 Different Features between Ruby and Python

② The Way to Cache Return Value via Delayed Evaluation

```

1 class User
2   You, 25 minutes ago • Commit sample code in Ruby
3   def initialize(name:, age:, occupation:, favourites: [])
4     @name = name
5     @age = age
6     @occupation = occupation
7     @favourites = favourites
8   end
9
10  def greet
11    "Hi, I'm #{@name} and #{@age} years old. \nMy occupation is #{@occupation}. \nMy favourite: #{inflect('language', @favourites.first)}."
12  end
13
14  private
15
16  attr_reader :name, :age, :occupation, :favourites
17
18  def favourite_programming_languages
19    @favourite_programming_languages ||= favourites.select { |f| f[:genre].equal?('programming') }
20  end
21
22  def favourite_programming_language_names
23    return 'nothing' if favourite_programming_languages.empty?
24    favourite_programming_languages.map { |it| it[:language] }.insert(-2, 'and').join(', ').gsub(/and\./, 'and')
25  end
26
27  def inflect(obj)
28    case obj
29    when 'be'
30      (0..1).include?(favourite_programming_languages.length) ? 'is' : 'are'
31    when 'language'
32      (0..1).include?(favourite_programming_languages.length) ? obj : "#{obj}s"
33    end
34  end
35
36  favourites = [
37    { genre: :programming, language: 'Ruby', level: 'Upper-Intermediate', years_of_experience: 7 },
38    { genre: :programming, language: 'Python', level: 'Intermediate', years_of_experience: 7 },
39    { genre: :programming, language: 'JavaScript', level: 'Intermediate', years_of_experience: 7 }
40  ]
41
42  user = User.new(name: 'Hayato Ishida', age: 33, occupation: 'Software Engineer', favourites: favourites)
43  puts user.greet

```

```

1 import re
2 You, 2 minutes ago | 1 author (You)
3 class User:
4     def __init__(self, name, age, occupation, favourites = []):
5         self.name = name
6         self.age = age
7         self.occupation = occupation
8         self.favourites = favourites
9         self.favourite_programming_languages = self._favourite_programming_languages_()
10
11     def greet(self):
12         return "Hi, I'm {name}-and {age}-years-old.\nMy occupation is {occupation}.\nMy favourite {language} {be_verb} {favourite_programming_language_names}.".format(
13             name = self.name,
14             age = self.age,
15             occupation = self.occupation,
16             language = self._infect__('language'),
17             be_verb = self._infect__('be'),
18             favourite_programming_language_names = self._favourite_programming_language_names_()
19         )
20
21     @private
22     def _favourite_programming_languages_(self):
23         self.favourite_programming_languages = []
24
25         for favourite in self.favourites:
26             if favourite['genre'] == 'programming':
27                 self.favourite_programming_languages.append(favourite)
28
29         return self.favourite_programming_languages
30
31     def _favourite_programming_language_names_(self):
32         if len(self.favourite_programming_languages) == 0:
33             return 'nothing'
34
35         favourite_programming_language_names = []
36
37         for favourite_programming_language in self.favourite_programming_languages:
38             favourite_programming_language_names.append(favourite_programming_language['language'])
39
40         favourite_programming_language_names.insert(-1, 'and')
41         return re.sub(r'and,', ', and', ','.join(favourite_programming_language_names))
42
43     def _infect__(self, obj):
44         match obj:
45             case 'be':
46                 if len(self.favourite_programming_languages) in range(0, 2):
47                     return 'is'
48                 else:
49                     return 'are'
50             case 'language':
51                 if len(self.favourite_programming_languages) in range(0, 2):
52                     return obj
53                 else:
54                     return '{obj}s'.format(obj = obj)
55
56 if __name__ == '__main__':
57     favourites = [
58         {'genre': 'programming', 'language': 'Ruby', 'level': 'Upper-Intermediate', 'years_of_experience': 7},
59         {'genre': 'programming', 'language': 'Python', 'level': 'Intermediate', 'years_of_experience': 7},
60         {'genre': 'programming', 'language': 'JavaScript', 'level': 'Intermediate', 'years_of_experience': 7}
61     ]
62
63     user = User(name = 'Hayato Ichida', age = 33, occupation = 'Software Engineer', favourites = favourites)
64     print(user.greet())

```

3. 3 Different Features between Ruby and Python

③ Receiver and Argument

I often get confused in which is receiver or argument when I code use some methods which has the same names and purposes in Ruby and Python in turns.

Array#join in Ruby and **String#join** in Python represent it.

- **Array#join** in Ruby: Receiver is the target elements in an array and argument is the delimiter.
- **String#join** in Python: Receiver is the delimiter in an array and argument is the target elements.

3. 3 Different Features between Ruby and Python

③ Receiver and Argument

```
1 class User
2   def initialize(name:, age:, occupation:, favourites: [])
3     @name = name
4     @age = age
5     @occupation = occupation
6     @favourites = favourites
7   end
8
9   def greet
10    "Hi, I'm-#{name}-and-#{age}-years-old.\nMy-occupation-is-#{occupation}.\nMy-favourite-#{inflect('language')}-#{inflect('be')}-#{favourite_programming_language_names}."
11  end
12
13  private
14
15  attr_reader :name, :age, :occupation, :favourites
16
17  def favourite_programming_languages
18    @favourite_programming_languages ||= favourites.select { |it| it[:genre].equal?(:programming) }
19  end
20
21  def favourite_programming_language_names
22    return 'nothing' if favourite_programming_languages.empty?
23    favourite_programming_languages.map { |it| it[:language] }.insert(-2, 'and').join(', ' ).gsub(/and\,/, 'and')
24  end
25
26  def inflect(obj)
27    case obj
28    when 'be'
29      (0..1).include?(favourite_programming_languages.length) ? 'is' : 'are'
30    when 'language'
31      (0..1).include?(favourite_programming_languages.length) ? obj : "#{obj}s"
32    end
33  end
34
35  favourites = [
36    {genre: :programming, language: 'Ruby', level: 'Upper-Intermediate', years_of_experience: 7},
37    {genre: :programming, language: 'Python', level: 'Intermediate', years_of_experience: 7},
38    {genre: :programming, language: 'JavaScript', level: 'Intermediate', years_of_experience: 7}
39  ]
40
41  user = User.new(name: 'Hayato-Ishida', age: 33, occupation: 'Software-Engineer', favourites:)
42  puts user.greet
```

```
1 import re
2
3 class User:
4     def __init__(self, name, age, occupation, favourites = []):
5         self.name = name
6         self.age = age
7         self.occupation = occupation
8         self.favourites = favourites
9         self.favourite_programming_languages = self.__favourite_programming_languages__()
10
11     def greet(self):
12         return "Hi, I'm-#{name}-and-#{age}-years-old.\nMy-occupation-is-#{occupation}.\nMy-favourite-#{language}-#{be_verb}-#{favourite_programming_language_names}.".format(
13             name = self.name,
14             age = self.age,
15             occupation = self.occupation,
16             language = self.__inflect__('language'),
17             be_verb = self.__inflect__('be'),
18             favourite_programming_language_names = self.__favourite_programming_language_names__()
19         )
20
21     # private
22
23     def __favourite_programming_languages__(self):
24         favourite_programming_languages = []
25
26         for favourite in self.favourites:
27             if favourite['genre'] == 'programming':
28                 favourite_programming_languages.append(favourite)
29
30         return favourite_programming_languages
31
32     def __favourite_programming_language_names__(self):
33         if len(self.favourite_programming_languages) == 0:
34             return 'nothing'
35
36         favourite_programming_language_names = []
37
38         for favourite_programming_language in self.favourite_programming_languages:
39             favourite_programming_language_names.append(favourite_programming_language['language'])
40
41         favourite_programming_language_names.insert(-2, 'and')
42         return re.sub(r'and,', 'and, ', ''.join(favourite_programming_language_names))
43
44     def __inflect__(self, obj):
45         match obj:
46             case 'be':
47                 if len(self.favourite_programming_languages) in range(0, 2):
48                     return 'is'
49                 else:
50                     return 'are'
51             case 'language':
52                 if len(self.favourite_programming_languages) in range(0, 2):
53                     return obj
54                 else:
55                     return '{obj}s'.format(obj = obj)
56
57 if __name__ == '__main__':
58     favourites = [
59         {'genre': 'programming', 'language': 'Ruby', 'level': 'Upper-Intermediate', 'years_of_experience': 7},
60         {'genre': 'programming', 'language': 'Python', 'level': 'Intermediate', 'years_of_experience': 7},
61         {'genre': 'programming', 'language': 'JavaScript', 'level': 'Intermediate', 'years_of_experience': 7}
62     ]
63     user = User(name = 'Hayato-Ishida', age = 33, occupation = 'Software-Engineer', favourites = favourites)
64     print(user.greet())
65
```

3. 3 Different Features between Ruby and Python

③ Receiver and Argument

They are interpreted as follows each.

- ``Array#join`` in Ruby: The target elements join together with the delimiter you provide.
- ``String#join`` in Python: The delimiter you provide joins the target element together.

In English, **given information already shared between the provider and the receiver is placed forward and new information not shared yet is backward.**

This rule is called **the End-Focus principle**.

That it to say, the information to be emphasise is placed at the end of the sentence.

3. 3 Different Features between Ruby and Python

③ Receiver and Argument

In the sense, when generating a string from the elements of an array, **Ruby is interested in delimiter and Python is in the elements of an array to join together.**

In terms of a developer's experience, it is more preferable to put the elements of an array first and the delimiter last, which enables us to make sense what joins together with what.

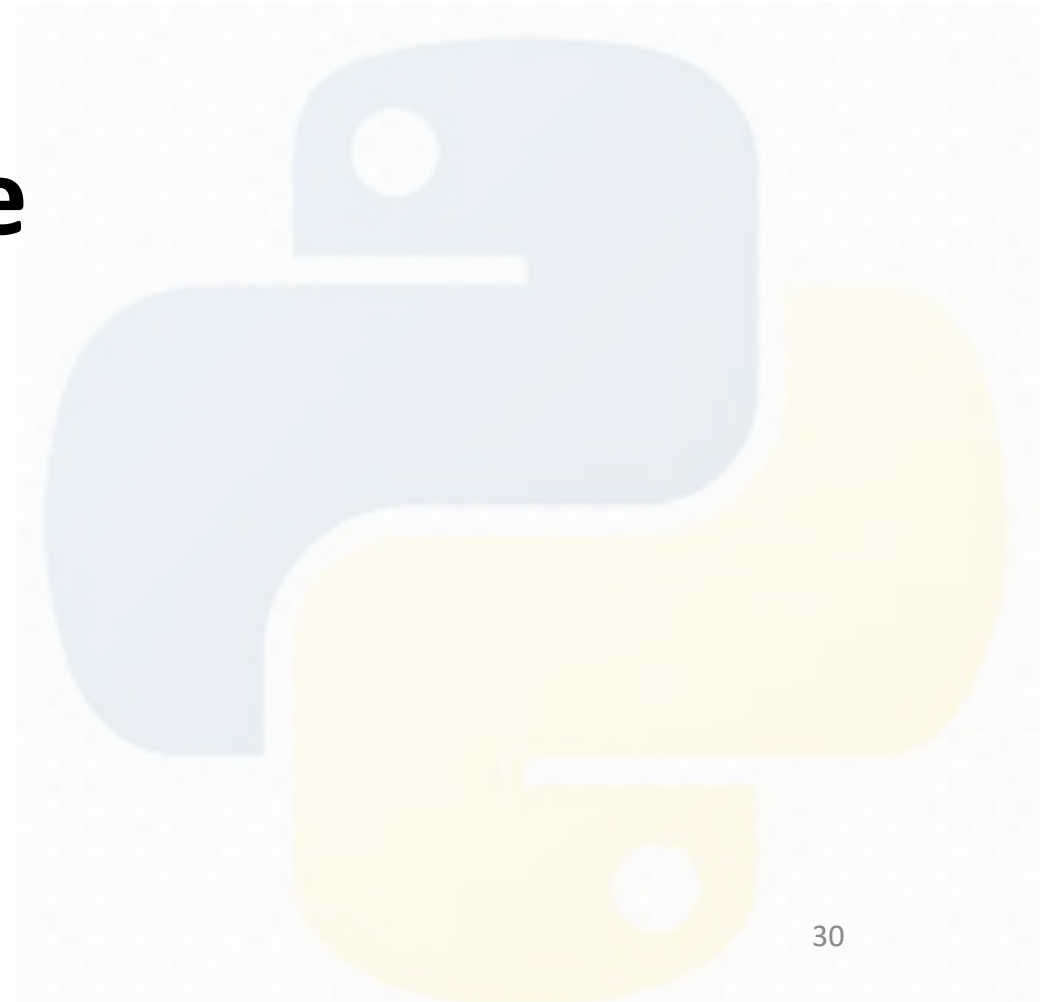
The difficulty in acquiring the second or a foreign language totally depends on what the mother tongue is just like natural languages, so I expressed it 'personal opinion'.

However, I am quite sure that **Ruby gives us a lot closer writing experience to English of the natural languages as a person who majored in English linguistics than Python Does.**



5. Epilogue

vs.



5. Epilogue

Did I succeed in pulling a trigger to make you all aware of what features and natures Ruby, your languages, has by comparison with Python for the past 5 minutes.

[*97 Things Every Programmer Should Know > 43. Know Well More than Two Programming Languages*](#) suggests that we should learn other languages which have different paradigms.

To compare it to natural languages, it is similar for me, a native Japanese language speaker, to learn English, one of the Germanic languages. (Refer to [*Dive Deeper into Present-Day English Vol.1 -What English Is- > 2. Relation between Language Families and Learning Costs*](#) to make sure what I mean).

There are loads of differences between them in terms of grammar, vocabulary and collocation. The same thing applies to programming languages.

5. Epilogue

It may not be a programming language, but I often compose workflows of GitHub Actions in Shell Script.

It always turns out that writing it just like as Ruby is impossible with Object-Oriented philosophy.

It says, "Don't use array too much and focus on integrate simple procedures." This is because of the difference in paradigm.

I have not commanded Data-Oriented languages such as Go Lang and Rust; however, I would pick them up in the same topic once I acquire them.

If you know much about them, would you please share your thought via comparative linguistics?



6. References

vs.



6. References

- [97 Things Every Programmer Should Know](#)
 - Last Accessed On: 10 February 2025
- [He who knows no foreign language knows nothing of his mother tongue - La Academia Language School](#)
 - Last Accessed On: 10 February 2025
- [object-oriented programming \(OOP\)](#)
 - Last Accessed On: 10 February 2025
- [Compiler vs. Interpreter: 12 Critical Differences To Know](#)
 - Last Accessed On: 10 February 2025



EOD
vs.

